

Aula 2: Introdução à TypeScript

Estruturas Básicas e Tipagem

Prof. Ronaldo Borges

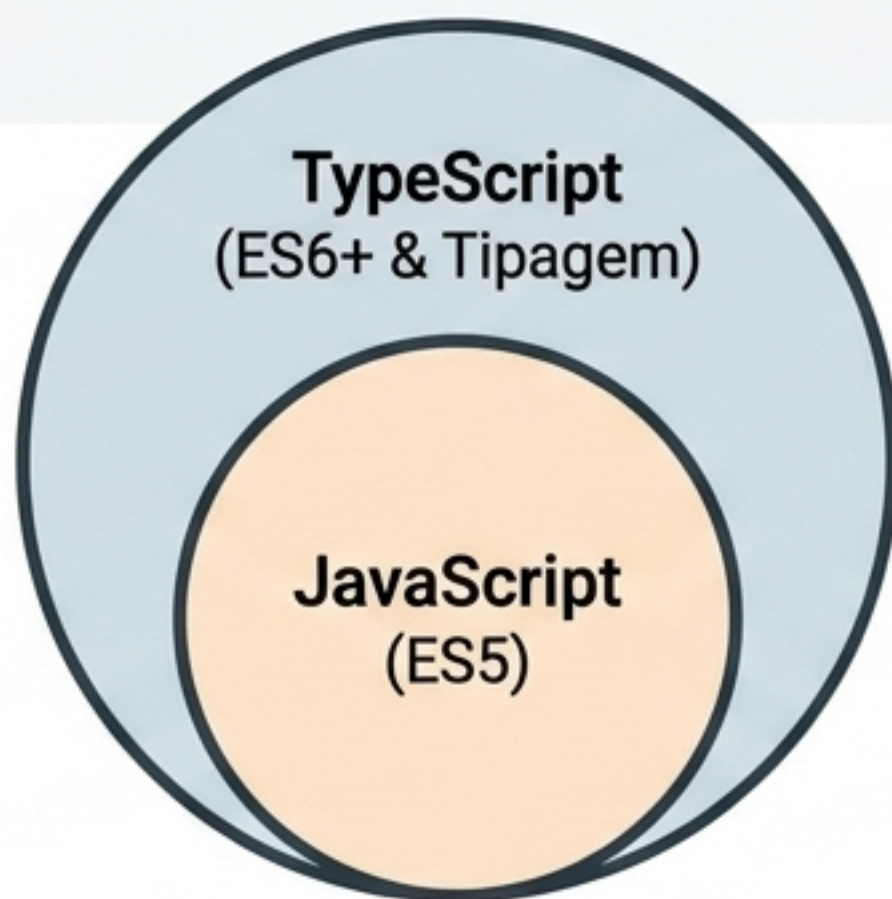
A Evolução da Web e o Problema do Escalonamento

- **A Origem:** O JavaScript foi criado em 1995 (Netscape) para adicionar pequenas dinâmicas a páginas HTML simples.
- **O Cenário Atual:** Hoje, roda aplicações corporativas complexas (Frontend e Backend via Node.js).
- **O Grande Problema:** A tipagem dinâmica permite que erros graves passem despercebidos durante o desenvolvimento.
- **A Consequência:** Erros de Runtime (Tempo de Execução). O sistema só quebra quando o usuário clica no botão, causando comportamentos inesperados.

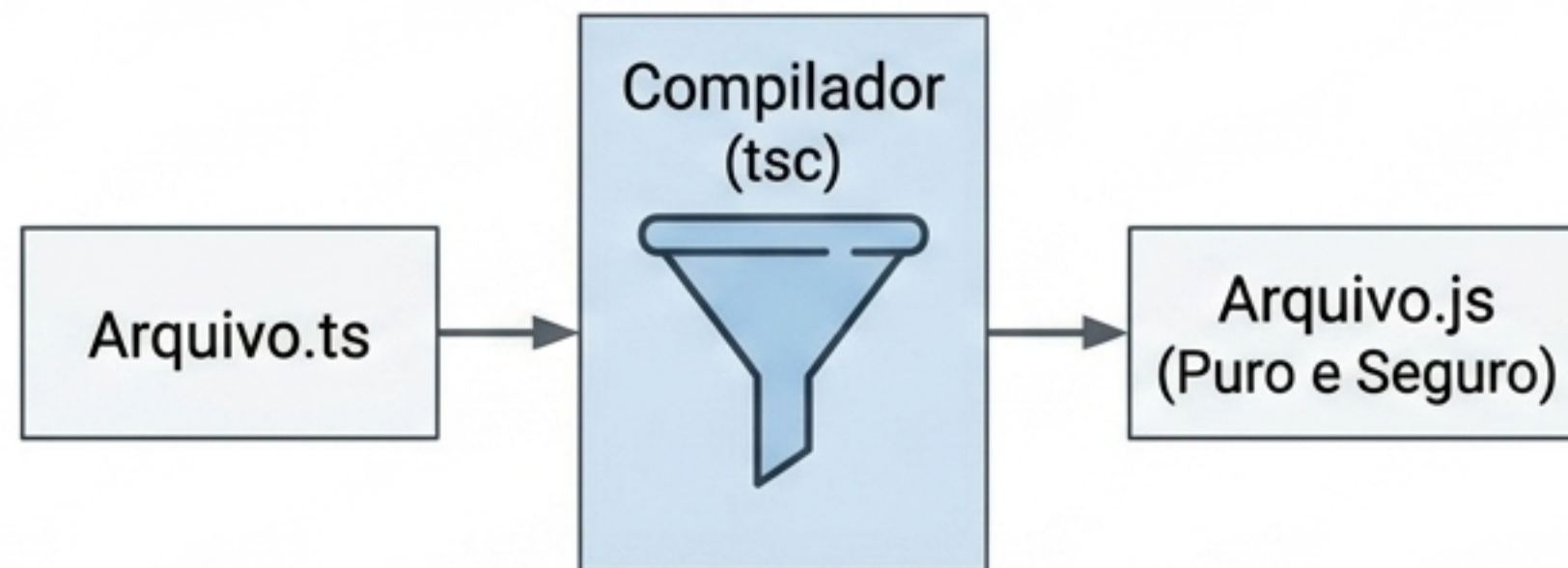


O que é TypeScript? (A Planta Arquitetônica)

Criado pela Microsoft (Anders Hejlsberg), o TypeScript é um Superset (superconjunto) sintático do JavaScript. Ele adiciona tipagem estática opcional (mas que usaremos de forma estrita) e recursos de orientação a objetos.



Todo código JavaScript válido é um código TypeScript



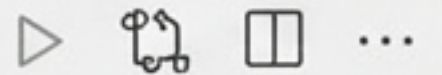
Matriz de Diagnóstico: JavaScript vs. TypeScript

Critério	JavaScript	TypeScript
Tipagem	Dinâmica (fraca) ❌	Estática (forte) ✅
Deteccção de Erros	Tempo de Execução (Runtime) ❌	Tempo de Compilação (Compile-time) ✅
Ecossistema	Navegador/Node nativo	Ferramenta de Desenvolvimento
Curva de Aprendizado	Menor (fácil início)	Maior (exige arquitetura)

O Superpoder: Checagem Estática na Prática

O TypeScript analisa seu código antes de rodar, identificando incompatibilidades matemáticas ou lógicas em tempo de edição.

TS exemplo.ts ×



TS exemplo.ts > ...

```
1 let pontuacao: number = 100;
```

```
2
```

Type 'string' is not assignable to type 'number'.

```
3 // O sistema tenta atualizar a pontuação com um texto
```

```
4 pontuacao = "cem";
```


Declaração de Variáveis: Abandone o var

A regra de ouro: O formato moderno exige a palavra-chave (**let** ou **const**), o identificador, e OBRIGATORIAMENTE a anotação explícita de tipo (**: tipo**).

let

Valores **mutáveis**.
Podem ser reatribuídos durante a execução, mantendo a integridade do tipo.



```
let idade: number = 20;  
idade = 21; // Correto
```

const

Valores **imutáveis**. Uma vez atribuídos, **não podem ser substituídos**.
Torna o código mais **previsível**.



```
const nome: string = "João";  
nome = "Maria";
```

Cannot assign to 'nome' because it is a constant.

Anatomia da Tipagem: Primitivos Básicos

Number

Diferente do Java (que divide em int, float, double), no TS todo valor numérico é simplesmente **number**.

```
const notaFinal: number = 9.75;  
let quantidade: number = 40;
```

String

Representa textos, suportando aspas duplas, simples ou template literals (crases).

```
const aluno: string = "Maria da Silva";
```

Boolean

Tipo lógico que só admite **true** ou **false**. (Não aceita 1 ou 0 nativamente como em C).

```
let estaAprovado: boolean = true;  
estaAprovado = 1;
```

Type 'number' is not assignable to type 'boolean'.

Escopo de Bloco (Onde a variável vive)

Assim como no Java, no TypeScript (usando **let** e **const**), um bloco de código delimitado por chaves `{}` define o escopo (tempo de vida) da variável.

Escopo Global

Escopo do Bloco IF

```
let multiplicador: number = 2;

if (multiplicador === 2) {
  let resultado: number = 20; // Nasce aqui
}

// Fora do escopo
console.log(resultado);
```

Cannot find name 'resultado'.

Tipagem Explícita vs. Inferência de Tipos

Embora utilizemos anotações explícitas (obrigatórias em nossos exercícios), o compilador do **TypeScript** é inteligente o suficiente para **deduzir o tipo pela atribuição inicial (Inferência)**.

Explícito (Nosso Padrão)

```
let maximo: number = 100;  
maximo = "cem"; Type 'string' is not assignable to type 'number'.
```

Inferência (Automático)

```
let minimo = 0; // TS infere que é 'number'  
minimo = "zero"; Type 'string' is not assignable to type 'number'.
```


Operadores Aritméticos Tipados

Exige o tipo **number** no resultado:

+	(Adição)
-	(Subtração)
*	(Multiplicação)
/	(Divisão)
%	(Módulo)

```
const totalItens: number = 10;  
const caixas: number = 2;  
const itensPorCaixa: number = totalItens / caixas;
```

```
// 0 Erro Evitado (0 que o JS legado faria):  
const falha: number = "10" / 2;
```

The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.

Operadores Relacionais e Lógicos

Relacional

Comparação direta. Sempre retorna boolean. Prefira `===` (Igualdade Estrita) e `!==` para evitar coerção implícita indesejada. Demais: `>`, `<`, `>=`, `<=`

```
const ehMaior: boolean = idade >= 18;
```

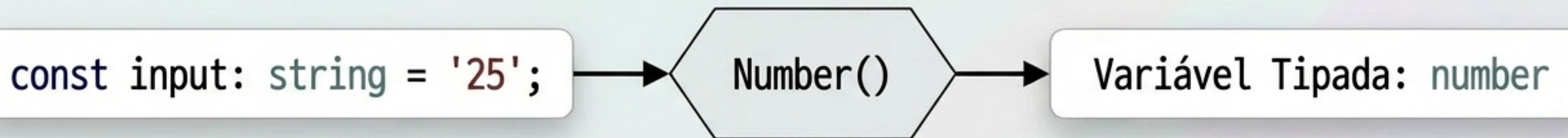
Lógico

Composição de expressões lógicas: `&&` (E), `||` (OU), `!` (NÃO).

```
const temIdade: boolean = idade >= 18;  
const temIngresso: boolean = true;  
const podeEntrar: boolean = temIdade && temIngresso;
```


Coerção Segura de Tipos (Casting)

Diferente da coerção implícita falha do JS, no TypeScript forçamos a conversão de forma explícita e mapeada para garantir compatibilidade.

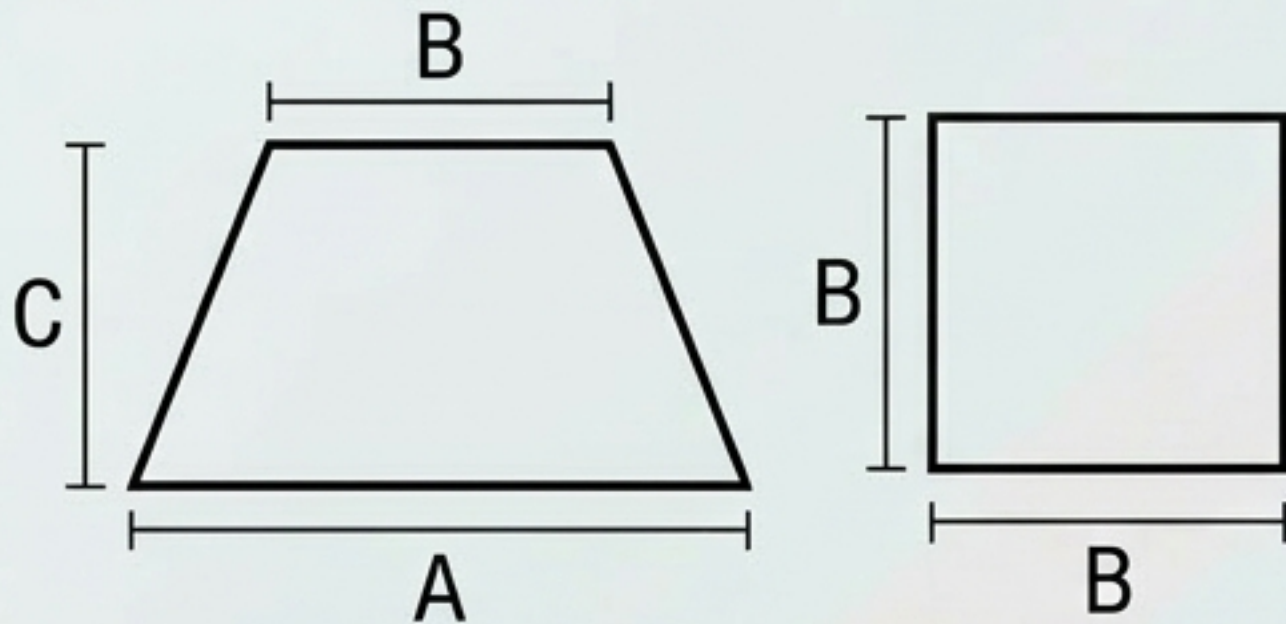


```
const inputUsuario: string = "150";  
  
// Erro de tipagem direta (evitado):  
// const peso: number = inputUsuario;  
  
// Casting seguro correto:  
const peso: number = Number(inputUsuario);  
const preco: string = String(19.99);
```


Síntese Prática: Geometria Tipada

Problema:

Escrever algoritmos para calcular a área de um trapézio (A: base maior, B: base menor, C: altura) e de um quadrado (lado B).



```
// Cálculo do Trapézio  
const baseMaior: number = 10;  
const baseMenor: number = 5;  
const alturaTrap: number = 6;
```

```
const areaTrapezio: number =  
    ((baseMaior + baseMenor) *  
    alturaTrap) / 2;
```

```
// Cálculo do Quadrado reaproveitando a  
base menor  
const ladoQuad: number = baseMenor;  
const areaQuadrado: number = ladoQuad *  
ladoQuad;
```


Referências Bibliográficas

BORGES, Ronaldo. *Java - Estruturas Sequenciais*. Material de Aula (Slides). Instituto Federal do Piauí - IFPI, Campus Floriano, 2025.

GRONER, Loiane. *Estruturas de dados e algoritmos com JavaScript*. São Paulo: Novatec Editora, 2019.

MICROSOFT. *TypeScript Language Specification: Version 0.9.1*. Microsoft Corporation, 2013. Disponível em: openwebfoundation.org.

UZAYR, Sufyan Bin. *TypeScript for Beginners: The Ultimate Guide*. [S.l.]: CRC Press, 2022.